AI GIS Road & Sidewalk Assessment Platform

Final technology stack with project-specific reasons | Free and open-source-first MVP

01 FRONTEND

TECHNOLOGY	WHY WE USE IT IN THIS PROJECT
React + Vite + TypeScript	Build a responsive, component-based web application. Vite gives fast local development; TypeScript catches type errors.
Tailwind CSS + shadcn/ui	Create consistent forms, navigation, dashboards, and accessible UI components without designing everything from scratch.
React Router + TanStack Query	Handle page navigation, API requests, caching, loading states, and refetching project/finding data.
React Hook Form + Zod	Build project and review forms with reusable field validation and clear feedback.
MapLibre GL JS	Display interactive survey maps, road layers, finding pins, and synchronized map selections.
Recharts	Display dashboard metrics such as processed images and reviewed findings.

02 BACKEND

TECHNOLOGY	WHY WE USE IT IN THIS PROJECT
Python + FastAPI	Expose REST APIs for projects, uploads, findings, reviews, inspection tracking, and reporting; integrate naturally with Python AI code.
Pydantic	Validate incoming API data and shape outgoing responses, including GPS metadata, filters, and reviewer decisions.
SQLAlchemy	Define and query related database records such as projects, images, findings, and review history using Python.
Alembic	Version and safely apply changes to database tables as new features are introduced.
PyJWT + pwdlib + custom RBAC	Implement login tokens and password hashing, plus server-side checks so users only access authorized organizations/projects.
Celery + Valkey	Queue time-consuming image analysis and report jobs; execute them through workers with status tracking and retries.
Pillow + OpenCV	Inspect photo format and dimensions, create image previews, and prepare files for AI inference.

03 AI / COMPUTER VISION

TECHNOLOGY	WHY WE USE IT IN THIS PROJECT
PyTorch + Torchvision	Train or fine-tune computer-vision models without depending on a proprietary AI API.
Faster R-CNN (baseline); compare RetinaNet	Start with a standard object detector and benchmark accuracy and speed on the pilot road-defect dataset before selecting the final model.
OpenCV + Pillow	Decode, preprocess, resize, and inspect road images for inference and quality checks.
NumPy + Pandas	Clean annotations, examine datasets, and generate training/evaluation summaries.
scikit-learn + TorchMetrics	Measure per-class precision/recall and other evaluation results on held-out images.
CVAT (self-hosted)	Allow qualified reviewers to label potholes, cracks, and pavement deterioration in training images.

04 DATABASE

TECHNOLOGY	WHY WE USE IT IN THIS PROJECT
PostgreSQL	Store organizations, memberships, projects, image metadata, AI findings, reviews, jobs, and inspections in related tables.
PostGIS	Store project boundaries and observation points; calculate distances and query nearby roads or facilities.
GeoAlchemy2	Use PostGIS geographic fields and spatial operations from SQLAlchemy models and backend queries.
Alembic	Apply reviewed schema migrations consistently in development, staging, and production.
PostgreSQL metadata + private image storage	Keep file IDs, GPS and access metadata in the database; store original photo bytes separately in private object storage (SeaweedFS proposed).

Licensing and cost note: Software is selected to avoid mandatory proprietary licenses. Hosting, storage, GPU hardware and bandwidth may still cost money. Verify the license of each pretrained model, dataset, and geographic data source before commercial deployment.